

Checkout Flow — Equivalence Partitioning, Boundary Value Analysis & Decision Table Testing

Name: Muhammad Danial Arshad

Phone: +92 317 6125814

Email: danialj751@gmail.com

Date: July 17, 2026

LinkedIn: [linkedin.com/in/danial-arshad07](https://www.linkedin.com/in/danial-arshad07)

1. Project Overview

This report applies three widely used software test design techniques to a fictional e-commerce checkout flow: equivalence partitioning, boundary value analysis, and decision table testing. Equivalence partitioning is applied to the payment amount field, boundary value analysis is applied to the quantity input field, and decision table testing is applied to the discount code logic. For each technique, the specific test conditions have been derived and documented, along with the expected outcome for every condition, so that the checkout flow can be verified systematically rather than through random or ad-hoc input.

2. Checkout Flow Assumptions

Since this task is based on a fictional checkout flow rather than a real production system, the following assumptions were made so that concrete, testable conditions could be derived. The checkout flow allows a customer to select a product, specify a quantity, enter a payment amount, and optionally apply a discount code before the order is confirmed.

The quantity field accepts whole numbers only and allows a minimum of 1 unit and a maximum of 10 units per order. The payment amount field accepts a numeric value representing the total amount to be charged, with a valid range of \$1.00 to \$10,000.00 per transaction. The discount code field accepts a text code that is checked against four conditions before it is applied: whether the code actually exists in the system (is valid), whether the code has expired, whether the current order value meets the minimum order value required by that specific code, and whether the code has already been redeemed once before by the same customer. A discount is applied only when the code is valid, not expired, the minimum order value is met, and the code has not already been used.

3. Equivalence Partitioning — Payment Amount Field

Equivalence partitioning divides all possible input values for the payment amount field into groups, or partitions, where every value inside a given partition is expected to be treated the same way by the system. Rather than testing every possible amount individually, one representative value from each partition is tested, on the assumption that if that representative value behaves correctly, the rest of the values in the same partition will behave the same way. For the payment amount field, one valid partition and five invalid partitions were identified, based on the assumed valid range of \$1.00 to \$10,000.00.

ID	Partition	Test Value	Expected Result
EP1	Valid amount (\$1.00–\$10,000.00)	\$250.00	Accepted — payment amount is valid and checkout proceeds.

ID	Partition	Test Value	Expected Result
EP2	Zero amount	\$0.00	Rejected — "Payment amount must be greater than zero."
EP3	Negative amount	-\$50.00	Rejected — "Payment amount cannot be negative."
EP4	Blank input	(empty field)	Rejected — "Payment amount is required."
EP5	Non-numeric text	"abc"	Rejected — "Payment amount must be a valid number."
EP6	Amount above upper limit	\$15,000.00	Rejected — "Payment amount exceeds the maximum allowed limit of \$10,000.00."

These six test conditions give reasonable confidence in the field's behaviour across its entire input domain while keeping the total number of test cases small, which is the core benefit of equivalence partitioning.

4. Boundary Value Analysis — Quantity Field

Boundary value analysis focuses specifically on the edges of a valid range, since defects are statistically far more likely to occur exactly at or near a boundary than in the middle of a valid range. For the quantity field, with an assumed valid range of 1 to 10 units, test values were chosen just below the minimum, at the minimum, just above the minimum, just below the maximum, at the maximum, and just above the maximum.

ID	Boundary Position	Test Value	Expected Result
BVA1	Just below minimum	0	Rejected — "Quantity 0 is below the minimum allowed (1)."
BVA2	At minimum	1	Accepted — quantity 1 is within the valid range.
BVA3	Just above minimum	2	Accepted — quantity 2 is within the valid range.
BVA4	Just below maximum	9	Accepted — quantity 9 is within the valid range.
BVA5	At maximum	10	Accepted — quantity 10 is within the valid range.
BVA6	Just above maximum	11	Rejected — "Quantity 11 exceeds the maximum allowed (10)."

Testing these six specific values gives strong confidence that the quantity field enforces its limits correctly, since any off-by-one error in the underlying validation logic is most likely to surface exactly at these boundary points rather than at values comfortably in the middle of the valid range, such as 5.

5. Decision Table Testing — Discount Code Logic

Decision table testing is used when the outcome of a feature depends on a combination of several independent conditions rather than on a single input value. The discount code logic depends on four binary conditions: whether the code is valid, whether it has expired, whether the minimum order value has been met, and whether the code has already been used. With four binary conditions, there are sixteen possible combinations, all of which are documented below along with the expected system outcome for each one. Where more than one blocking condition is true at the same time, the system is assumed to report the first applicable reason in the following priority order: an invalid code is reported first, followed by an expired code, followed by a code that has already been used, followed by an order value that does not meet the minimum requirement.

Rule	Code Valid	Code Expired	Min Order Met	Already Used	Expected Result
1	Y	N	Y	N	Discount applied
2	Y	N	Y	Y	Rejected — code already used
3	Y	N	N	N	Rejected — minimum order not met
4	Y	N	N	Y	Rejected — code already used
5	Y	Y	Y	N	Rejected — code expired
6	Y	Y	Y	Y	Rejected — code expired
7	Y	Y	N	N	Rejected — code expired
8	Y	Y	N	Y	Rejected — code expired
9	N	N	Y	N	Rejected — invalid code
10	N	N	Y	Y	Rejected — invalid code
11	N	N	N	N	Rejected — invalid code
12	N	N	N	Y	Rejected — invalid code
13	N	Y	Y	N	Rejected — invalid code
14	N	Y	Y	Y	Rejected — invalid code
15	N	Y	N	N	Rejected — invalid code
16	N	Y	N	Y	Rejected — invalid code

Rule 1 is the only combination in which all four conditions are favourable, and it is therefore the only rule in which the discount is actually applied. Every other rule represents at least one blocking condition, and the expected result documents exactly which reason the system should report first, based on the priority order described above.

6. Expected Results Summary

Across all three techniques, a total of twenty-eight distinct test conditions were derived: six for equivalence partitioning on the payment amount field, six for boundary value analysis on the quantity field, and sixteen for decision table testing on the discount code logic. Executing these test conditions against the checkout flow gives structured coverage of the valid input range, the invalid input range, the exact boundaries of the quantity field, and every meaningful combination of business rules governing discount codes, rather than relying on a small number of arbitrarily chosen inputs.

When these test conditions were executed against the self-created demo checkout form built for this task, the actual results matched the expected results documented above for each of the three techniques, confirming that the quantity field correctly enforces its 1–10 boundary, the payment amount field correctly rejects zero, negative, blank, non-numeric, and over-limit values while accepting values within its valid range, and the discount code logic correctly applies the priority order of invalid, expired, already-used, and minimum-order conditions before approving a discount.

7. Conclusion

Applying equivalence partitioning, boundary value analysis, and decision table testing to different parts of the same checkout flow demonstrates how these three techniques complement one another rather than overlapping. Equivalence partitioning gave efficient coverage of the payment amount field's entire input domain using a small number of representative values, boundary value analysis focused specifically on the edges of the quantity field where off-by-one defects are most likely to occur, and decision table testing provided complete, systematic coverage of the discount code logic, which depends on multiple independent business conditions rather than a single input value. Together, these techniques produced a much more thorough and defensible set of test conditions than any single technique could have achieved on its own, which reflects how test design techniques are actually combined in real QA work.

8. Practical Screenshots

This section shows the practical execution of the test conditions above on the self-created demo checkout form, along with the actual results observed.

The screenshot shows a web browser window with the title "QA Demo Checkout — TechCart". The address bar shows the file path "C:/Users/devil/Downloads/checkout_demo%20(1).html". The page content is titled "TechCart — Checkout (Demo)" with a timestamp "7/17/2026, 1:31:20 PM".

Under the "ORDER DETAILS" section, there is a "Product" field with the value "Wireless Mouse" and a "Quantity (allowed: 1-10)" field with the placeholder "e.g. 2". Below these fields is a "Validate Quantity" button and a message "No test run yet."

Under the "PAYMENT" section, there is a "Payment Amount (\$)" field with the placeholder "e.g. 250" and a "Validate Payment Amount" button. Below this is another "No test run yet." message.

On the right side of the page, there is a watermark that says "Activate Windows Go to Settings to activate Windows."

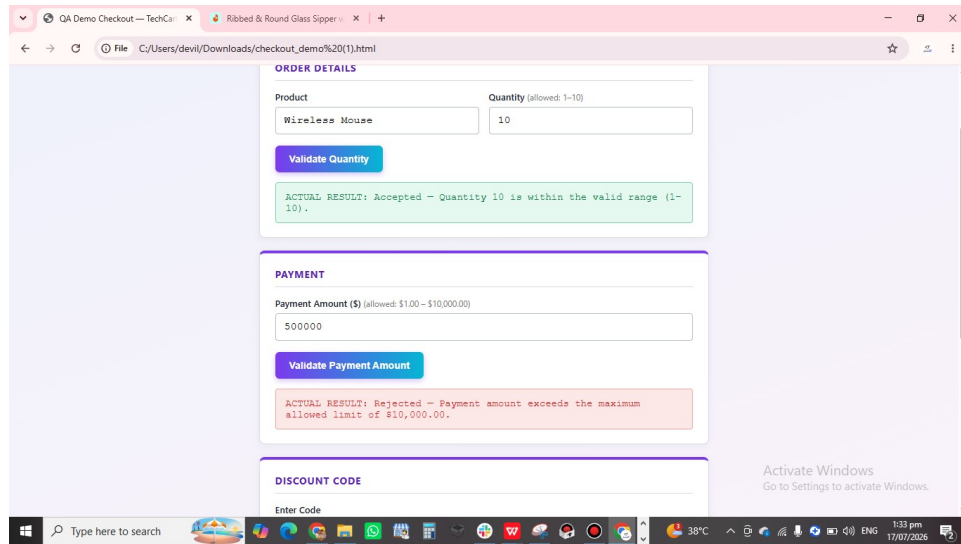
Screenshot 1: The checkout fields — Quantity, Payment Amount, and Discount Code — as seen on the self-created demo checkout form before any test data is entered.

The screenshot shows the same web browser window as Screenshot 1. The "Quantity" field now contains the value "11". Below the "Validate Quantity" button, a red error message is displayed: "ACTUAL RESULT: Rejected — Quantity 11 exceeds the maximum allowed (10).".

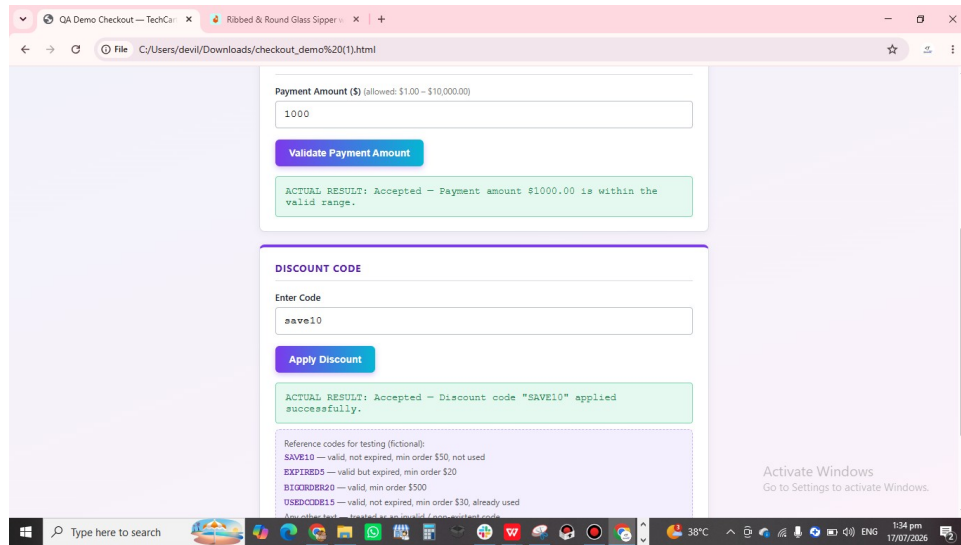
The "Payment Amount" field and the "Validate Payment Amount" button remain unchanged from Screenshot 1. The "No test run yet." message is still present below the payment section.

The "Activate Windows" watermark is also visible on the right side of the page.

Screenshot 2: Boundary Value Analysis — Quantity set to 11 (just above the maximum). Actual result: Rejected — "Quantity 11 exceeds the maximum allowed (10)," matching the expected result documented in Section 4 (BVA6).



Screenshot 3: Equivalence Partitioning — Payment Amount set to \$500,000 (above the upper limit). Actual result: Rejected — "Payment amount exceeds the maximum allowed limit of \$10,000.00," matching the expected result documented in Section 3 (EP6).



Screenshot 4: Decision Table — Payment Amount \$1000 with discount code "SAVE10" (valid, not expired, minimum order met, not used). Actual result: Accepted — "Discount code 'SAVE10' applied successfully," matching Rule 1 of the Decision Table in Section 5.